

WS & SOA (IT644) — Project Plan

PrepNova

RAG-based Adaptive Mock Interview Platform — 14-Week Project Plan

GitHub Repository: github.com/Krishna4801/PrepNova

Team Members

Name	Technical Role
Hardi Dhadhal	Frontend 1 — Authentication, Dashboard & General UI
Shruti Makwana	Frontend 2 — Interview Interface & Interview Flow
Varunee Agrawal	Backend 1 — Authentication, User & Resume/JD APIs
Kush Dani	Backend 2 — Interview, Evaluation, Report APIs & Database
Krishna Shah	AI 1 — Resume/JD Processing, Embeddings & RAG
Garv Modi	AI 2 — Retrieval, Answer Evaluation, Adaptive Scoring & Vector DB
Dev Patel	AI + Frontend — Question Generation & AI-Powered Interview Experience
Jaimin Trivedi	AI + Backend — AI Service & Backend-AI Orchestration

1. Project Description

PrepNova is an AI-powered web application that provides personalized mock interview practice using Retrieval-Augmented Generation (RAG). It supports two interview modes: a resume-based mode, where questions are grounded in the candidate's own resume (optionally combined with a job description), and a role-based mode, where questions are drawn from a curated role knowledge base when no resume is uploaded (optionally also combined with a JD).

After the interview, PrepNova produces a performance report with an AI-based score out of 10, category-wise breakdown, strengths and weaknesses, JD skill-gap analysis (when a JD was used), and personalized improvement suggestions, all stored in the candidate's interview history.

2. Application Type

PrepNova is a browser-based web application following a client-server architecture: a React frontend, a Node.js/Express backend, and a Python AI/RAG service, communicating over REST APIs.

3. System Overview & Interview Modes

3.1 Mode 1 — Resume-Based Interview

The user uploads a resume, which is parsed, chunked, embedded, and stored in the vector store. The user may optionally also provide a Job Description (JD); when present, the system compares the candidate's profile against the JD's requirements to generate job-specific questions rather than only resume-specific ones (e.g. probing a JD-required skill like Kubernetes that isn't evidenced in the resume).

3.2 Mode 2 — Role-Based Interview

If no resume is provided, the user selects a target role and the system retrieves context from the curated role knowledge base instead. A JD can still optionally be added here too, narrowing a generic role interview into one specific to a real job opening.

3.3 Input Combination Rules

Resume	Role	JD	Interview Context Used
Provided	Optional	Not provided	Resume
Not provided	Selected	Not provided	Role Knowledge Base
Provided	Optional	Provided	Resume + JD
Not provided	Selected	Provided	Role + JD
Not provided	Not selected	Not provided	Not enough information — interview cannot start

A resume tells the system what the candidate has done; a role tells it what type of job they're preparing for; a JD tells it what a specific company actually expects. Combining resume + JD lets PrepNova identify matched, weak, and missing skills and target questions accordingly — this JD-aware skill matching is one of PrepNova's key differentiators over a static question bank.

4. High-Level Architecture

The system is organized into four layers: the React frontend, the Node.js/Express backend (with JWT auth and MongoDB), the Python AI/RAG service (FastAPI), and the vector store used for semantic retrieval over resumes, JDs, and role knowledge. The AI service handles PDF/JD processing, chunking, embeddings, retrieval, question generation, and answer evaluation, and returns structured results back through the backend to the frontend.

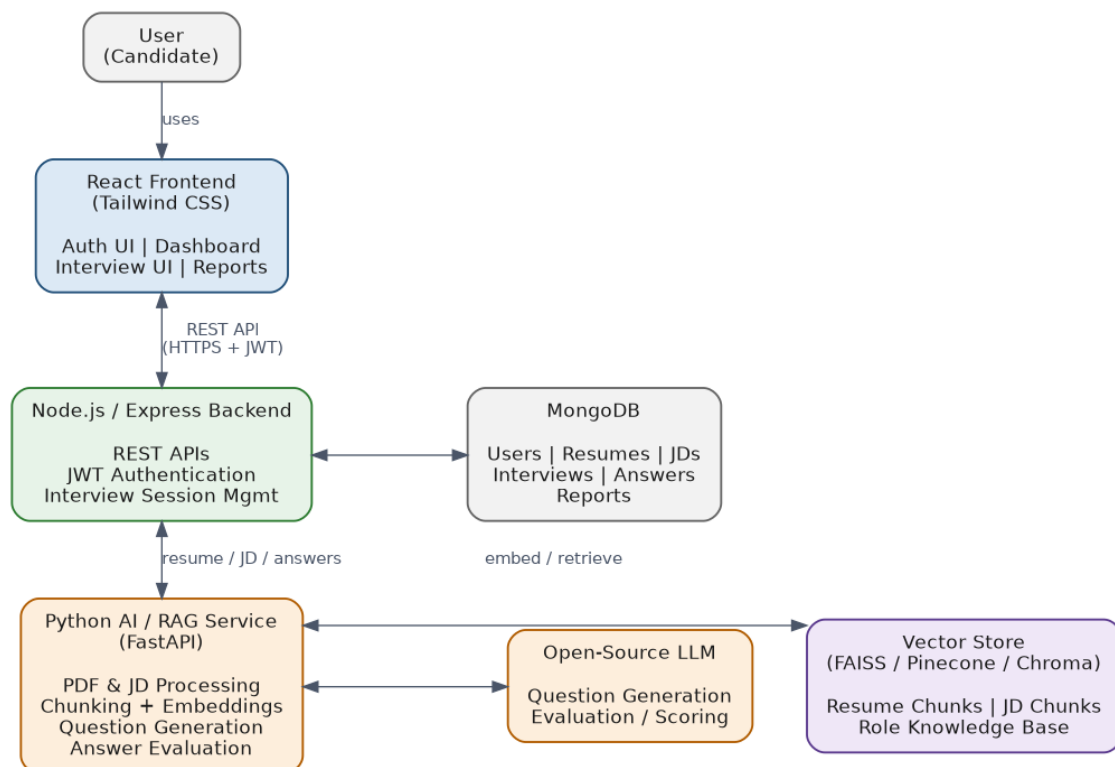


Figure 1: PrepNova high-level system architecture — request/response flow between the frontend, backend, AI/RAG service, database, vector store, and LLM.

4.1 Core System Components

Component	Main Responsibility
React Frontend	User interaction across auth, dashboard, and interview screens
Authentication (JWT)	Identity and access control
Node.js Backend	API and application orchestration
Resume Service	Resume ingestion
JD Service	Job-description ingestion
PDF Parser	Extract resume text (with OCR fallback for scans)
Chunker	Split resume/JD into retrievable, metadata-tagged chunks
Embedding Service	Convert chunks into vectors
Vector Store	Semantic storage and search
Role Knowledge Base	General role knowledge for the no-resume path
Retrieval Engine	Retrieve relevant context for a given query/question
Skill Matcher	Identify JD-vs-resume alignment (matched/weak/missing)
Question Generator	Generate personalized questions across categories and difficulty
Difficulty Engine	Control Easy/Medium/Hard, incl. JD-aware adjustment
Interview Session Manager	Manage interview lifecycle/state
Answer Manager	Store candidate responses with full grounding context
Evaluation Engine	Evaluate answers against a fixed rubric
Feedback Generator	Produce strengths/weaknesses/suggestions
Report Generator	Build the final performance report
History Manager	Track previous interviews for progress tracking
MongoDB	Persistent application data

5. Technology Stack

5.1 Explicitly Defined by the Project

- Frontend: React.js, Tailwind CSS, JavaScript
- Backend: Node.js, Express.js, REST APIs
- AI/RAG Layer: Python (RAG pipeline, embeddings, model integration)
- Database: MongoDB
- Authentication: JWT
- AI Model: Open-source LLM (specific model to be finalized)
- Vector Store: candidates are Pinecone / FAISS / ChromaDB (to be finalized)
- Version Control: Git & GitHub

5.2 Recommended Implementation Choices

Area	Recommendation	Reasoning
Vector Store	FAISS (dev) → Pinecone (production, optional)	Free, local, no cloud dependency for a student project
Embedding Model	BAAI/bge-small-en-v1.5 (or bge-base for higher quality)	Strong open-source retrieval-quality/cost tradeoff
LLM	Qwen2.5-7B-Instruct or an equivalent open-weight instruct model	Runs at 7–8B scale, good instruction-following for question/eval tasks
PDF Extraction	PyMuPDF (fitz)	Lightweight and fast for text-based PDFs
OCR Fallback	Tesseract or an OCR-capable model	Handles scanned/image-based resumes
AI Service API	FastAPI	Clean Python service boundary for Node ↔ AI communication
RAG Orchestration	LangChain or a custom Python pipeline	Either is workable; custom gives more control for a class project
Data Validation	Pydantic	Enforces structured JSON from the LLM (question/evaluation schemas)

6. Interview Question Types

Questions are generated along two independent dimensions: a content category (what the question is about) and a structural format (how it's presented and scored). Every generated question carries one category and one format, tagged with a difficulty level.

6.1 Question Categories (Content)

#	Category	Description
1	Resume-Based	Generated directly from the candidate's resume content
2	JD-Based	Generated from job-description requirements
3	Role-Based	Generated from the curated role knowledge base
4	Technical Concept	Tests understanding of a core technical concept
5	Project-Based	Explores a specific project listed on the resume
6	Skill-Based	Tests a specific skill the candidate has claimed
7	Scenario-Based	Presents a realistic problem or situation to solve
8	Problem-Solving	Requires working through a technical problem
9	System Design	Tests architecture and design thinking at scale
10	Debugging	Presents a bug or issue to identify and fix
11	Behavioral / HR	Explores soft skills via past behavior (STAR format)
12	Situational	"What would you do if..." hypothetical question
13	Skill-Gap	Targets a skill required by the JD but missing/weak in the resume
14	Resume + JD Combined	Cross-references a resume strength against a JD requirement
15	Follow-Up / Cross-Question	Digs deeper based on the candidate's previous answer
16	Experience-Based	Explores the depth of claimed work experience
17	Real-World Application	Tests the ability to apply theory practically

#	Category	Description
18	Communication / Explanation	Tests the ability to explain a concept clearly
19	Leadership & Teamwork	Explores collaboration or leadership experience
20	Difficulty-Based	Easy / Medium / Hard tagged variant of any category above

6.2 Question Formats (Structure)

Format	Description	Primary Use
MCQ	Single-correct-answer multiple choice, 4 options	Quick concept/knowledge check
Short Answer (1–2 lines)	Brief factual or definitional response	Technical recall, terminology
Scenario / Subjective	Open-ended problem or design question requiring explanation	Applied understanding, reasoning
Coding / Debugging	Small code-writing, debugging, or problem-solving task	Practical technical skill
Behavioral / HR (STAR)	Situation-Task-Action-Result style question	Communication, soft skills
Resume/JD Deep-Dive	Open-ended question grounded in a specific resume or JD chunk, or a follow-up/cross-question	Validates real experience and probes deeper

6.3 Question Mix by Difficulty Level

Difficulty	MCQ	Short Answer	Scenario	Coding/Debug	Behavioral	Resume/JD Deep-Dive
Easy	20%	20%	15%	15%	10%	20%
Medium	15%	15%	20%	15%	15%	20%
Hard	5%	15%	20%	15%	20%	25%

Difficulty additionally reacts to JD skill matching, and — from Week 10 — to the adaptive difficulty engine, which adjusts the next question's difficulty based on the previous answer's score.

7. Evaluation Methodology

Each answer is evaluated on a fixed rubric — Relevance, Technical Correctness, Completeness, Clarity, and Depth — rather than a single unexplained score, and the evaluator model always receives the same retrieved grounding context that was used to generate the question, so feedback stays tied to actual facts instead of generic commentary.

7.1 Scoring Method per Format

Format	Scoring Method	Notes
MCQ	Exact match, auto-graded	Binary: full or zero marks, instant
Short Answer	Embedding cosine similarity vs. model answer + keyword check	Threshold-based partial credit
Scenario / Subjective	LLM-as-judge against the 5-part rubric	Relevance, correctness, completeness, clarity, depth

Format	Scoring Method	Notes
Coding / Debugging	Test-case execution where feasible; otherwise LLM code-review rubric	Correctness, efficiency, readability
Behavioral (STAR)	LLM checks STAR-structure completeness + clarity	Verifies all 4 STAR components are present
Resume/JD Deep-Dive	LLM-as-judge, re-grounded in the original retrieved chunk	Flags vague or inconsistent answers vs. stated experience

7.2 Example Evaluation Output

Rather than returning only a number, the evaluator returns a structured object such as:

```
{ "score": 7, "relevance": 8, "correctness": 7, "completeness": 6, "clarity": 8,
"strengths": [...], "weaknesses": [...], "suggestions": [...] }
```

7.3 Aggregation into the Final Report

- Technical Score — MCQ, Short Answer, Coding, and technical Scenario questions
- Problem-Solving Score — Coding/Debugging and Scenario questions specifically
- Communication Score — Behavioral questions and answer clarity across all formats
- Resume/JD Alignment Score — Deep-dive and skill-gap questions, plus the JD skill-matching output (matched / weak / missing skills)
- From Week 9: an ATS-Lite Score — keyword match between the resume and the target role's expected skill list — is also shown alongside the interview report

8. Testing Strategy

PrepNova is validated across 15 testing types spanning individual components, cross-module integration, AI-specific quality checks, and full end-to-end acceptance against the project's requirements. Testing runs alongside development throughout all 14 weeks rather than as a single end-of-project phase — the specific unit-test or integration-check task for a given week is included directly in that week's individual member tasks in Section 13, and Weeks 11–12 add dedicated CI/CD, automated E2E, and performance-testing passes.

Testing Type	What to Test	Main Features
Unit Testing	Individual functions/components	PDF parsing, chunking, API functions, scoring, validation
Integration Testing	Multiple components working together	Frontend ↔ Backend, Backend ↔ MongoDB, Backend ↔ AI service, RAG pipeline
System / End-to-End Testing	Complete application workflow	Login → Resume/JD → Interview → Answers → Report
Functional Testing	Whether each feature works to requirements	Resume upload, role selection, JD, difficulty, questions, reports
API Testing	REST API correctness	Auth, resume, JD, interview, report APIs
Database Testing	Data storage/retrieval correctness	Users, resumes, JD, interviews, answers, reports
AI/RAG Testing	Quality of retrieval and generated questions	Embeddings, retrieval, context grounding, question generation
AI Evaluation Testing	Quality of answer evaluation	Score, strengths, weaknesses, feedback
Security Testing	Authentication and authorization	JWT, user-data isolation, unauthorized API access
Performance Testing	Speed and behavior under load	API response, RAG retrieval, concurrent users

Testing Type	What to Test	Main Features
UI Testing	Frontend behavior	Upload, interview screen, dashboard, report
Usability Testing	Ease of use	Interview flow, error messages, navigation
Regression Testing	New changes don't break old features	Entire application after updates
Error / Negative Testing	Invalid inputs and failure scenarios	Corrupt PDF, empty JD, invalid login, API failures
Acceptance Testing	Meets final project requirements	Complete PrepNova functionality

9. Edge Case Handling

Situation	Detection	Handling
Unsupported resume file	File validation	Reject with clear error
Corrupted PDF	Parser failure	Show error, request re-upload
Empty resume	No meaningful extracted text	Reject / request another file
Scanned resume	Insufficient extractable text	OCR fallback
Empty JD	Text validation	Reject with message
Very short JD	Low text length	Accept, limited personalization
Very large JD	Length threshold	Chunk and retrieve relevant portions
Duplicate JD	Hash comparison	Reuse already-processed JD
No resume provided	Resume absent	Fall back to role-based mode
No role selected (no resume)	Role absent	Require role selection
No resume and no role	Missing context	Do not allow interview to start
Missing JD skill	Skill matching	Ask a foundational/knowledge-check question
Embedding failure	Model error	Retry with backoff
Vector store unavailable	Connection error	Retry, then fail gracefully
LLM unavailable / timeout	Timeout or error response	Retry, then show a clear error
Duplicate question generated	Similarity check	Regenerate a different question
Empty answer submitted	Input validation	Reject or record as no-response
Invalid or expired JWT	Auth check fails	401 Unauthorized, redirect to login
Unauthorized interview access	user_id mismatch	403 Forbidden
Interview abandoned mid-session	Session timeout / user exits	Mark as abandoned, preserve partial answers
Report generation failure	Evaluation error	Preserve stored answers, retry generation

10. Work Distribution Among 8 Technical Roles

Rather than splitting work purely by proposal section, PrepNova is divided into 8 technical roles across four layers — Frontend, Backend, AI, and two AI-integration bridge roles — so each member owns a role they can explain independently, while staying tightly connected to the roles on either side of them.

Frontend 1 — Hardi Dhadhal

Authentication, Dashboard & General UI

- Login/register pages, protected routes, auth state, form validation
- Dashboard: previous interviews, stats, average/best score, start-new-interview
- Resume upload UI, JD input/paste UI, role & difficulty selection UI
- Navigation, sidebar, shared components, loading/error states, responsive layout

Frontend 2 — Shruti Makwana

Interview Interface & Interview Flow

- Interview screen: question, answer input, submit, next question
- Interview state: current question, progress, difficulty, remaining questions
- Question display for technical/HR/resume/JD/role/scenario/follow-up types
- Answer interface, interview completion flow, interview history UI

Backend 1 — Varunee Agrawal

Authentication, User & Resume/JD APIs

- Register/login, password hashing, JWT generation/verification, auth middleware
- User profile APIs; resume APIs (upload/get/delete); JD APIs (create/get/delete); role APIs
- Authorization, user-data isolation, input & file validation, API security

Backend 2 — Kush Dani

Interview, Evaluation, Report APIs & Database

- Interview lifecycle APIs (start/answer/next/finish); interview state model
- AI-service communication for question/answer/evaluation flow
- Report APIs, history APIs, aggregation queries
- Database management across all collections

AI 1 — Krisha Shah

Resume/JD Processing, Embeddings & RAG

- PDF parsing/OCR, text cleaning, section-aware chunking for resume and JD
- Embedding pipeline and vector-store population with metadata
- RAG quality evaluation: relevance, duplication, hallucination checks

AI 2 — Garv Modi

Retrieval, Answer Evaluation & Adaptive Scoring

- Vector store integration and retrieval logic (query → embed → top-K search) for question-generation and evaluation context
- Answer evaluation: correctness, relevance, completeness, clarity, depth
- Adaptive difficulty logic and ATS-lite scoring
- Prompt engineering and AI-quality testing for retrieval and evaluation prompts

AI + Frontend — Dev Patel

Question Generation & AI-Powered Interview Experience

- Question generation across all 20 content categories and 3 difficulty levels; context-aware generation using resume + JD + role + retrieved context + history
- Dynamic question/follow-up experience and AI loading states
- AI error handling in the UI (timeout, invalid response, network failure)
- Streaming/real-time response display
- Interactive report visualization

AI + Backend — Jaimin Trivedi

AI Service & Backend–AI Orchestration

- Python AI service (FastAPI) bridging Node backend and the AI pipeline
- RAG orchestration: retrieve → build prompt → call LLM → validate → return
- LLM integration: config, token limits, timeout/retry handling
- AI response validation and deployment coordination

Ownership & Collaboration Summary

Layer	Roles	What They Own
Frontend	Frontend 1, Frontend 2	Everything the user sees: auth, dashboard, and the live interview experience
Backend	Backend 1, Backend 2	Application logic, REST APIs, and all persistent data
AI / RAG	AI 1, AI 2	Resume/JD understanding, retrieval, question generation, and evaluation
Integration Bridges	AI + Frontend, AI + Backend	Connect the AI layer to the live UI and to the backend service layer

11. Timeline with Milestones (14 Weeks)

The project runs across a 14-week schedule — an 8-week core-platform build followed by a 6-week extended-features, hardening, and testing phase — sized against a 200 person-hour workload per team member (Section 12).

Week	Milestone
Week 1	Requirement analysis, literature survey, project planning, UI design
Week 2	Database design, backend setup, REST API development, authentication
Week 3	Resume upload, PDF parsing, text chunking, embedding pipeline setup
Week 4	Vector store integration, RAG retrieval logic, role-based knowledge base

Week	Milestone
Week 5	Dynamic question generation across difficulty levels, mock interview flow
Week 6	AI-based evaluation, scoring, and feedback report generation
Week 7	Dashboard, interview history, integration testing, bug fixing
Week 8	MVP stabilization — regression fixes from Week 7 testing, internal checkpoint demo
Week 9	Automatic role detection from resume + lite ATS compatibility scoring module
Week 10	Real-time adaptive difficulty (adjusts per answer score) + curated question bank expansion
Week 11	CI/CD pipeline setup (automated test runs on PRs) + security hardening
Week 12	Automated E2E test suite (Cypress) + performance/load testing on RAG retrieval and core APIs
Week 13	Accessibility pass, UI polish from testing findings, code documentation
Week 14	Final regression testing, production deployment, final documentation, presentation preparation

12. Estimated Effort per Member

Effort is tracked by phase against the 200 person-hour target per member for this course. Numbers reflect implementation, testing, documentation, and coordination time for that phase, not just feature-building time.

Member	Wk1	Wk2-6	Wk7-8	Wk9-10	Wk11-12	Wk13	Wk14	Total
Varunee	8	64	24	24	44	20	18	202
Hardi	8	64	24	24	32	30	18	200
Krishna	8	74	24	24	32	20	18	200
Shruti	8	64	33	24	32	20	18	199
Kush	8	64	24	24	45	20	18	203
Jaimin	8	64	24	24	32	20	33	205
Dev	8	72	24	24	32	20	18	198
Garv	8	64	24	35	32	20	18	201

Wk2-6 covers core platform build (backend, frontend, RAG pipeline); Wk7-8 covers integration testing and stabilization; Wk9-10 covers extended features (auto role detection, ATS-lite scoring, adaptive difficulty); Wk11-12 covers CI/CD, security hardening, automated E2E, and load testing; Wk13 covers accessibility and documentation; Wk14 covers final regression, deployment, and submission. Individual totals range 198–205 hours, consistent with the 200-hour-per-member requirement.

13. Individual Team Member Timeline

The milestones above are broken down into individual weekly tasks for each member below. Unit tests for a given feature are written by the same member, in the same week the feature is built, rather than being deferred.

Week 1 — Requirement Analysis & Planning

Member	Task
Varunee	Plan authentication flow and JWT structure
Hardi	UI/UX mockups, design system, resume-upload wireframe
Krishna	Research resume parsing & chunking approaches; shortlist library; finalize vector store choice
Shruti	Wireframe interview-interface and dashboard screens

Member	Task
Kush	Design database schema; plan REST API endpoints
Jaimin	Research evaluation/scoring approach; plan deployment pipeline; set up Git repository and task board
Garv	Research vector-DB retrieval strategies and answer-evaluation techniques; draft initial evaluation prompt templates
Dev	Study question-generation approaches and AI response formats; plan the dynamic question/follow-up experience with Frontend 2

Week 2 — Backend Setup, Database, Authentication

Member	Task
Kush	Implement database models and core CRUD APIs; write unit tests for models (Jest)
Varunee	Implement JWT authentication (login/register APIs); write unit tests for auth endpoints (Supertest)
Hardi	Build reusable UI components (buttons, forms, layout)
Shruti	Set up React project structure and routing
Krisha	Set up PDF parsing and text-extraction pipeline; provision vector store instance and test embedding storage
Garv	Test AI model API integration (retrieval and evaluation calls); prototype vector DB query interface
Dev	Test AI model API integration (question-generation call); prototype AI loading-state UI patterns (generating question / evaluating answer) for the interview screen
Jaimin	Set up staging deployment environment; run Week 2 integration check (register → login → JWT)

Week 3 — Resume Upload, Parsing, Embedding Setup

Member	Task
Krisha	Implement chunking strategy; generate resume embeddings; connect embeddings to vector store; write unit tests for chunking (Pytest)
Garv	Implement retrieval-query logic (embed → top-K search) against the vector store; draft curated role-based question bank (no-resume fallback) for Dev to consume
Hardi	Build resume-upload UI and connect to backend
Kush	Build resume-upload API endpoint; write unit tests for upload endpoint
Shruti	Build role-selection dropdown UI
Varunee	Secure file-upload endpoint; input validation
Dev	Build initial AI error-handling states in the UI (upload/parsing failures, timeouts)
Jaimin	Test resume-parsing edge cases and error handling; run Week 3 integration check (upload → chunks stored with correct metadata)

Week 4 — Vector Store, RAG Retrieval, Knowledge Base

Member	Task
Krishna	Optimize chunking/embedding quality; support vector-store indexing for faster retrieval; write unit tests for the embedding pipeline
Garv	Finalize retrieval logic (top-k context fetch) against the vector store; test retrieval accuracy against sample resumes; write unit tests for the retrieval function; build role-based curated question bank as retrieval fallback
Jaimin	Integrate RAG output into question-generation flow; run Week 4 integration check (query returns relevant chunks)
Kush	Build API to trigger RAG pipeline from frontend
Varunee	Secure RAG-related API endpoints
Shruti	Build difficulty-level selection UI
Hardi	Build interview question-display UI
Dev	Build the "retrieving context / generating question" loading UI tied to the RAG pipeline

Week 5 — Dynamic Question Generation & Interview Flow

Member	Task
Garv	Optimize retrieval latency; format retrieved context for use in question-generation and evaluation prompts; support Dev's question-generation integration
Jaimin	Test question-generation quality across difficulty levels; run Week 5 integration check (resume/role → question displayed)
Shruti	Build full interview flow (question → answer → next)
Hardi	Build response-input UI
Krishna	Monitor embedding/vector-store performance to support question-generation integration; assist with embedding edge-case testing
Dev	Implement question generation with difficulty-level tuning; write unit tests for prompt construction; implement dynamic follow-up UI flow (question → answer → AI follow-up → display), working with Frontend 2
Kush	Build interview-session API (start/submit/next question)
Varunee	Session management and auth checks on interview APIs

Week 6 — AI Evaluation, Scoring, Feedback Report

Member	Task
Jaimin	Build evaluation/scoring logic (score out of 10); write unit tests including malformed-response fallback case
Garv	Build feedback generation (strengths/weaknesses); retrieve evaluation context needed for scoring accuracy
Kush	Build report-storage API
Varunee	Secure report-related APIs
Krishna	Test evaluation output on sample resumes; monitor embedding/vector-store stability for the evaluation pipeline
Shruti	Build report/score display page

Member	Task
Hardi	Design report visualization UI (score, feedback cards); run Week 6 integration check (submit answer → score returned)
Dev	Implement the "evaluating answer" loading state and score/feedback reveal on submission

Week 7 — Dashboard, History, Integration Testing

Member	Task
Shruti	Build dashboard and interview-history UI
Hardi	Polish UI/UX and responsiveness across all pages
Kush	Build interview-history/tracking APIs
Varunee	End-to-end auth and security testing
Krishna	Integration testing of the parsing/embedding pipeline
Garv	Integration testing of the retrieval logic and answer-evaluation pipeline
Dev	Integration testing of question generation and the AI-to-UI flow across interview and report screens; bug fixing
Jaimin	Full cross-module integration testing and regression testing of Week 2–6 features; coordinate bug fixes

Week 8 — MVP Stabilization & Checkpoint

Member	Task
Varunee	Fix auth/security issues found during Week 7 integration testing
Hardi	Fix UI issues found during Week 7 testing; prepare internal checkpoint demo walkthrough
Krishna	Fix parsing/embedding issues found during Week 7 testing
Shruti	Fix interview-flow issues found during Week 7 testing
Kush	Fix backend/API issues found during Week 7 testing
Jaimin	Coordinate bug triage from Week 7 integration testing; run internal checkpoint demo
Garv	Fix retrieval and answer-evaluation issues found during Week 7 testing
Dev	Fix question-generation and AI-UI integration issues found during Week 7 testing (loading states, follow-up flow)

Week 9 — Auto Role Detection & ATS-Lite Scoring

Member	Task
Krishna	Implement keyword/skill extraction from resume text to auto-detect a likely target role
Garv	Extend retrieval logic to use the auto-detected role when none is manually selected; build ATS-lite scoring logic — keyword match between resume and the role's expected skill list, weighted scoring
Jaimin	Write tests for ATS scoring edge cases (empty resume, no matching keywords)
Kush	Build ATS score API endpoint (GET /api/resume/:id/ats-score) and store score on the resume document
Varunee	Secure the new ATS endpoint; begin rate-limiting middleware groundwork for Week 11

Member	Task
Shruti	Build the ATS score display component on the report page
Hardi	Design the ATS score UI (score badge + keyword match breakdown)
Dev	Build the AI-driven UI feedback for the auto-detected role (e.g. "We detected this looks like an AI Engineer profile"), with a loading state during detection

Week 10 — Adaptive Difficulty & Question Bank Expansion

Member	Task
Garv	Implement adaptive difficulty — next question's difficulty adjusts based on the previous answer's score (score $\geq 8 \rightarrow$ increase, $\leq 4 \rightarrow$ decrease); validate retrieval coverage across the expanded question bank
Jaimin	Update the evaluation flow to return the next difficulty alongside the score; unit tests for boundary cases
Krishna	Generate and store embeddings for the newly authored curated questions; verify embedding quality for the new vector-store entries
Kush	Update the interview-session API to support dynamic difficulty progression per question
Varunee	Update session schema and auth checks for the adaptive flow
Shruti	Update the interview-flow UI to reflect difficulty changes between questions
Hardi	Update the question-display UI with a per-question difficulty indicator
Dev	Author additional curated questions across roles/difficulties to expand the fallback question bank; implement the real-time UI transition when adaptive difficulty changes the next question (indicator/animation), working with Frontend 2

Week 11 — CI/CD Pipeline & Security Hardening

Member	Task
Krishna	Add automated tests for the chunking/embedding pipeline into the CI pipeline
Kush	Set up a GitHub Actions workflow to run the Jest/Supertest suite on every PR to the backend
Varunee	Implement rate limiting (express-rate-limit) on auth and resume-upload endpoints; audit input sanitization across all POST endpoints
Jaimin	Add an automated staging-deployment step to the CI/CD pipeline on merge to main
Garv	Add automated tests for the retrieval pipeline into CI; review prompt-injection risk in RAG prompts and sanitize resume text before it is inserted into prompts
Shruti	Set up frontend lint and build verification checks in CI
Hardi	Cross-browser and responsive layout check across all pages; document fixes needed
Dev	Add frontend-side validation guards for malformed or unsafe AI response rendering

Week 12 — Automated E2E & Performance Testing

Member	Task
Shruti	Write a Cypress E2E test for the core flow: register $\rightarrow$ resume upload $\rightarrow$ start interview $\rightarrow$ answer $\rightarrow$ view report
Hardi	Write a Cypress E2E test for the role-based (no-resume) flow and dashboard/history navigation

Member	Task
Kush	Run load testing (Locust/k6) against the interview-session and evaluation APIs; document throughput/latency
Krishna	Run load testing against the embedding/vector-store pipeline; identify and document bottlenecks under concurrent load
Varunee	Load-test auth endpoints; verify rate limiting holds under load
Jaimin	Compile E2E and load-test results into a test report; triage any blocking issues found
Garv	Run load testing against the retrieval endpoint; identify and document bottlenecks under concurrent load; fix retrieval and answer-evaluation issues surfaced by E2E testing
Dev	Fix question-generation and AI-to-UI integration issues surfaced by E2E testing (loading states, follow-up rendering)

Week 13 — Accessibility, Documentation & Polish

Member	Task
Hardi	Accessibility pass — alt text, colour contrast, keyboard navigation across all pages
Shruti	UI polish based on Week 12 E2E and load-test findings
Krishna	Write code documentation (docstrings) for the resume parsing and embedding modules
Kush	Write code documentation (JSDoc) for backend controllers
Varunee	Write code documentation (JSDoc) for auth middleware; finalize API documentation (Postman collection / OpenAPI spec)
Garv	Write code documentation for the retrieval and answer-evaluation modules
Jaimin	Write code documentation for the evaluation module; consolidate documentation into the repo's /docs folder
Dev	Write code documentation for the question-generation module; accessibility review of AI-driven UI states (loading, streaming, error messages) — screen-reader announcements for dynamic content

Week 14 — Final Regression, Deployment & Presentation

Member	Task
Jaimin	Final production deployment and deployment documentation; run a deployment smoke test
Varunee	Final production security check (auth, API access, rate limits)
Kush	Final backend documentation compilation
Krishna	Final embedding & vector-store documentation compilation
Garv	Final retrieval and evaluation documentation compilation
Shruti / Hardi	Final presentation slide design and demo script
Dev	Final question-generation documentation compilation; final polish of AI-driven UI states; support final presentation demo/script
All members	Final full-system regression testing; final report compilation and submission

14. Project Management Tools

The team uses Trello to track tasks and manage the workflow, and a Gantt-style timeline (Figure 2) for the overall schedule view.

14.1 Task Board Structure (Trello)

- Backlog — tasks not yet started
- To Do — tasks planned for the current week
- In Progress — tasks currently being worked on
- In Review — completed tasks pending review/testing by another member
- Done — completed and verified tasks

Each card is tagged with the responsible member's name, the relevant week, and a label indicating the module (Frontend / Backend / AI-RAG). Cards are created directly from the individual weekly task breakdown in Section 13, including the testing tasks.

14.2 Timeline Chart

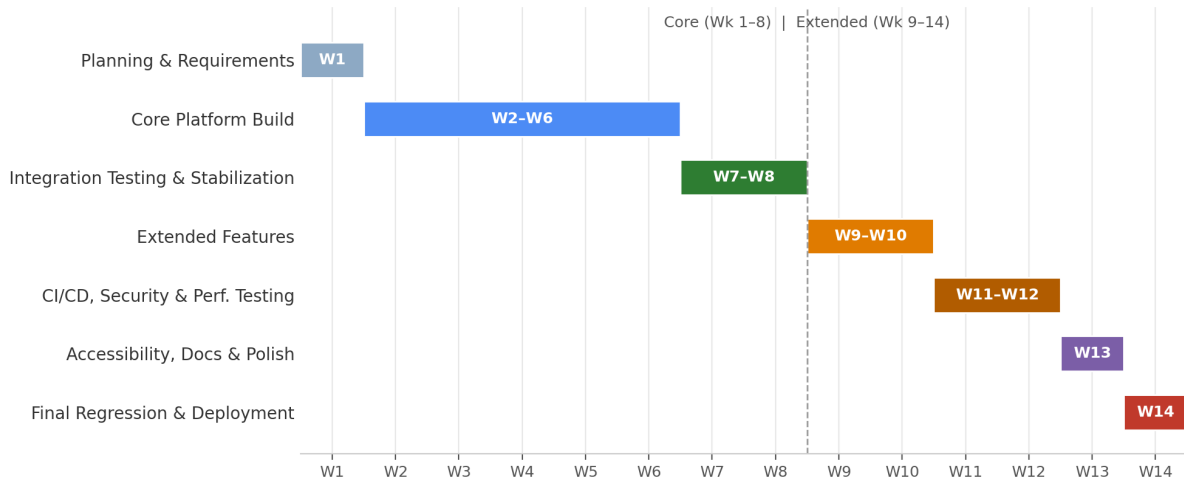


Figure 2: 14-week phase timeline — 8-week core platform build followed by a 6-week extended-features, hardening, and testing phase.